***Register the DataFrame in DuckDB:*** DuckDB can query Polars DataFrames directly with zero-copy Arrow integration:

```
import duckdb

con = duckdb.connect()
con.register(“student_features”, features)
```

We now have an SQL table backed by Polars’ high-performance memory layout.

***Run analytics with SQL:*** How does sleep affect performance?

```
sleep_stats = con.execute(“””SELECT
        sleep_category,
        COUNT(*) AS students,
        ROUND(AVG(performance), 2) AS avg_performance,
        ROUND(AVG(performance_per_hour), 2) AS efficiency
    FROM student_features
    GROUP BY sleep_category
    ORDER BY avg_performance DESC
“””).fetchdf()
Sleep_stats
```

This provides a clear understanding of how sleep quality relates to academic performance and efficiency.

## Best practices and pitfalls

Despite their efficiency, Polars and DuckDB are constrained by single-machine memory limits. Engineers should avoid eager materialisation of large intermediate results and rely on Polars’ lazy execution as long as possible. A common Polars pitfall is calling *.collect()* too early, which forces execution and materialises data unnecessarily. On the DuckDB side, large joins or aggregations can spill to disk if memory limits are exceeded, which may surprise users expecting purely in-memory execution.

Understanding execution boundaries, such as when Polars plans execute and when DuckDB runs eagerly, is essential for building predictable, efficient pipelines.

For teams adopting the Polars and DuckDB combination, a pragmatic approach is to replace local Spark jobs or Pandas-based pipelines incrementally, using Polars for transformations and DuckDB for analytical queries. Beyond performance, the real win is developer experience: faster iteration, simpler deployment, and analytics pipelines that fit naturally into modern application code. END


**By: Aanchal Narendran**

The author is working as a developer in the AI centre of excellence at IBM.